

ChromoVM: A Persistent Decentralized RISC-V Runtime Anchored to Ethereum

Technical Whitepaper v0.1

August 2026

Abstract

Blockchains provide a durable and economically secured state machine, but the same properties that make consensus useful also make it inefficient as a general application runtime. Most decentralized applications therefore place nearly every authoritative state transition inside a block, forcing application latency, throughput, and execution cost to inherit the constraints of the underlying chain.

ChromoVM separates execution from settlement. Applications execute as persistent RISC-V processes on an off-chain network of staked nodes, while Ethereum is used for identity, staking, balances, checkpoint commitments, dispute settlement, and execution of authorized on-chain actions. A program may continue running and exchanging messages without waiting for a new Ethereum block. When stronger assurance is required, the program can request a checkpoint whose state transition is secured by a staked executor quorum, an optimistic proof-on-challenge mechanism, or an immediate validity proof.

The system does not introduce a second blockchain. There is no global block producer, global replicated state, or independent consensus ledger in the execution layer. Each application has its own ordered input stream, machine state, and checkpoint sequence. Computation scales horizontally across programs, while Ethereum remains the common trust and asset layer.

This paper describes the execution machine, deterministic runtime, process and message model, node network, storage, state commitments, adaptive assurance, Ethereum bridge, resource accounting, staking, and native protocol economics.

1. Introduction

A smart contract is a useful mechanism for shared state whose correctness must be agreed upon by a decentralized network. It is not, however, an efficient replacement for a continuously running application server. Programs that need event loops, long computations, frequent state updates, large memory, background jobs, or interactive low-latency behavior either pay the cost of executing these operations on-chain or move them to ordinary centralized infrastructure.

The common compromise is therefore split trust: assets and final settlement are placed on a blockchain, while most useful application logic is executed by a server controlled by the application operator. This restores performance but weakens the trust model precisely where application behavior becomes complex.

ChromoVM is designed around a narrower question:

Can an application execute continuously outside Ethereum, while allowing selected state transitions to inherit Ethereum-backed assurance when the application actually needs it?

The answer proposed here is a persistent deterministic runtime. Application code is compiled to a standard RISC-V target and executed by a network of staked nodes. The execution layer is intentionally not a blockchain. It provides computation, ordering, replication, storage, and availability. Ethereum provides the protocol registry, node collateral, program balances, checkpoint anchoring, dispute settlement, and custody of on-chain assets.

The key distinction is that assurance belongs to a transition rather than to the entire application. A game movement, feed update, background calculation, or chat message may use a fast committee receipt. A

marketplace settlement may use optimistic assurance. A large withdrawal may require an immediate validity proof. The same application may use all three.

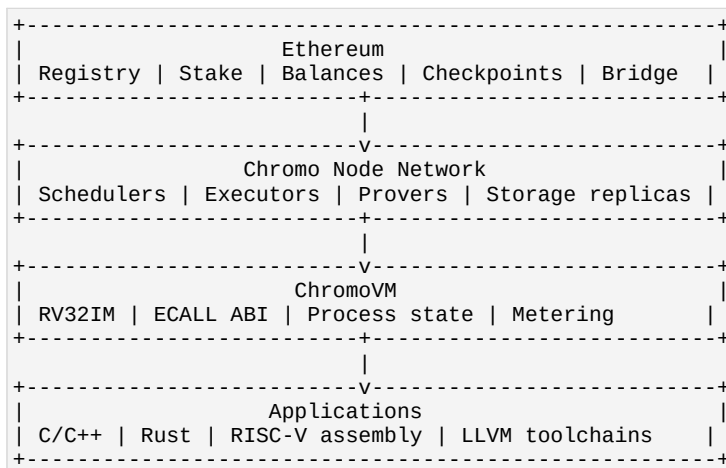
2. Design Goals

ChromoVM is designed under the following constraints.

1. General application execution. Programs should not be restricted to the programming model of the EVM.
2. Existing toolchains. The execution ISA should already have compilers, assemblers, linkers, debuggers, and a mature specification.
3. Determinism. The same machine state and the same ordered inputs must always produce the same resulting state and outputs.
4. Persistent processes. A program should be able to yield, wait for input, and later resume from the exact machine state at which it stopped.
5. Low-latency execution. Ordinary application progress should not require an Ethereum transaction or block for every state transition.
6. Ethereum-backed settlement. High-value transitions and on-chain actions must be able to reach an assurance level whose correctness is enforceable on Ethereum.
7. Horizontal scaling. Independent applications should not contend for a single global execution sequence.
8. Minimal new consensus. The system should reuse Ethereum rather than create an independent blockchain.
9. Metered resource use. Compute, storage, bandwidth, and proof work must be measurable and payable.
10. Replaceable components. Networking, storage, proving, and local databases are implementation components and should be replaceable without changing the logical protocol.

3. System Model

The system consists of four logical layers.



Ethereum is authoritative for protocol membership, collateral, balances, settled checkpoint status, and execution of Ethereum action intents.

The node network is authoritative only within the assurance level selected by the program. Fast receipts are economically trusted committee statements. Optimistic checkpoints become final if not successfully challenged. Proven checkpoints are accepted only with a valid execution proof.

ChromoVM is the deterministic machine whose transition function is shared by every implementation.

4. The Execution Machine

4.1 Canonical ISA

ChromoVM uses the RISC-V RV32IM instruction set as its canonical application ISA. RV32I provides a 32-bit integer machine with 32 integer registers, and the M extension adds integer multiplication and division.

RV32IM is selected instead of a custom CPU architecture for three reasons. First, existing C, C++, Rust, LLVM, GNU, ELF, and debugging toolchains can target it [1][2][3][4]. Second, the small integer ISA is practical to interpret, trace, meter, and prove. Third, existing RISC-V proving systems already use closely related execution models [8].

Floating-point, vector, privileged, atomic, and host-specific instructions are not part of the canonical Chromo execution contract. Floating-point behavior needed by applications should use deterministic software libraries unless a future protocol version explicitly standardizes additional instructions.

The canonical machine is little-endian and single-threaded per process. Parallelism is obtained by running many processes or many independent application actors rather than by exposing nondeterministic host threads inside one process.

4.2 Registers and Control State

A runnable process contains at minimum:

pc	32-bit program counter
x[0..31]	RISC-V integer registers
memoryRoot	commitment to linear memory
storeRoot	commitment to persistent key/value storage
inboxSeq	next input sequence number
outboxSeq	next output sequence number
status	RUNNABLE WAITING PAUSED EXITED FAULTED
units	cumulative compute units

Register x0 is always zero as required by RISC-V.

4.3 Executable Format

Applications are deployed as statically linked RISC-V ELF executables together with a manifest. Dynamic linking is excluded from the consensus execution path because dependency resolution must not depend on the executor host.

A deployment package contains:

```
code.elf
manifest.cbor
optional immutable assets
```

The package is content-addressed. Ethereum stores the program identifier, code hash, manifest hash or content identifier, owner, security policy, and initial state commitment rather than the full binary.

4.4 Execution Slices

A persistent application is not executed for an unbounded amount of time in a single protocol transition. Execution is divided into deterministic slices.

A slice is:

$$T(S_n, I_n, L_n) \rightarrow (S_{n+1}, O_n, U_n)$$

where:

S _n	initial machine state
I _n	ordered input batch
L _n	resource limit for the slice
O _n	ordered outputs
U _n	compute units consumed

Execution stops when one of the following occurs:

1. the program explicitly yields;
2. the program waits for input;
3. the program requests a checkpoint;
4. the slice resource limit is exhausted;
5. the program exits;
6. the machine traps or faults.

The resulting CPU state is committed, allowing the next slice to resume from the exact instruction and register state.

This converts a continuously running program into a sequence of bounded deterministic transitions without forcing each transition into an Ethereum block.

5. Chromo Runtime ABI

5.1 ECALL Boundary

Ordinary computation uses standard RISC-V instructions. Interaction with the decentralized runtime occurs through the RISC-V ECALL instruction.

The ABI defines a numeric syscall identifier and fixed argument/result conventions using RISC-V ABI registers. The runtime is therefore an execution environment rather than a custom CPU extension.

Initial syscall classes are:

```
PROCESS_YIELD
PROCESS_EXIT
MESSAGE_RECV
MESSAGE_SEND
STORE_GET
STORE_PUT
STORE_DELETE
BLOB_PUT
BLOB_GET
CHAIN_READ
CHAIN_ACTION
CHECKPOINT
INPUT_METADATA
LOG
```

A syscall is part of the deterministic state transition. Host operating system calls are never exposed directly to an application.

5.2 No Ambient Host State

The following values are not readable directly from the executor machine:

```
wall-clock time
local filesystem
host random generator
arbitrary network socket
environment variables
process identifiers
host CPU counters
```

Any external fact must enter the application as an explicit committed input.

A time value, for example, is an input carrying a protocol-defined timestamp. An HTTP result is an oracle or adapter message. An Ethereum observation is a chain input associated with a block reference and verification data. Randomness is a committed input from an approved source.

This rule is required for deterministic replay.

6. Persistent Processes and Messaging

6.1 Program Identity

Each deployed program receives a deterministic program identifier:

```
programId = keccak256(
  chainId ||
  registryAddress ||
  deploymentNonce ||
  owner ||
  codeHash
)
```

A program may own multiple processes. A process identifier is derived from the program and a monotonically increasing spawn nonce.

6.2 Input Messages

Users and other programs interact with a process by sending signed messages.

A canonical message contains:

```
destination
sender
senderNonce
payloadHash
payload
requestedAssurance
expiry
signature
```

A scheduler cannot fabricate a user's message because the executor verifies the sender signature. A scheduler can, however, delay or reorder messages within the ordering rules allowed by the program's policy. This is treated as a liveness and fairness problem, not a state-correctness problem.

6.3 Application-to-Application Messages

Program outputs may create messages for another Chromo process.

An inter-program message references:

```
sourceProgram
sourceCheckpoint
sourceOutboxSeq
destinationProgram
payloadHash
assuranceLevel
```

The destination specifies the minimum source assurance it accepts. A game may accept a FAST message from another game service, while a financial program may require the source message to originate from a PROVEN checkpoint.

6.4 Ordering

Chromo does not maintain a global transaction order.

Each program has its own ordered input batches. A scheduler assigned to the program forms a batch and signs its sequence. Executor committee members independently verify message signatures, nonce validity, batch continuity, and deterministic ordering constraints before executing it.

Programs therefore progress independently:

```
Program A: 1 -> 2 -> 3 -> 4 -> ...
Program B: 1 -> 2 -> ...
Program C: 1 -> 2 -> 3 -> ...
```

There is no requirement that A:4 and B:2 share a single global block height.

This is the principal source of horizontal scalability.

7. Node Network

7.1 Node Roles

A physical node may perform one or more roles.

Scheduler

Receives valid messages, constructs program input batches, and publishes ordered batch proposals.

Executor

Replays input batches, executes ChromoVM, produces receipts, and signs resulting state commitments.

Prover

Generates a validity proof for a requested transition.

Storage replica

Retains program packages, snapshots, input batches, and state data required for recovery and proof generation.

Challenger

Observes optimistic checkpoints and challenges a transition it believes to be invalid.

No role creates blocks for a new chain.

7.2 Networking

The reference network uses libp2p for peer identity, transport negotiation, encrypted connections, discovery, request/response protocols, and publish/subscribe propagation [6].

Typical protocol topics are scoped by program or role:

```
/chromo/program/<id>/messages
/chromo/program/<id>/batches
/chromo/program/<id>/receipts
/chromo/checkpoints
/chromo/nodes
```

Large state blobs are fetched by content identifier rather than flooded through gossip.

7.3 Committee Assignment

Ethereum maintains the active staked node set and provides the settlement security assumed by the protocol [5]. At the beginning of an assignment epoch, nodes are ranked for a program using deterministic rendezvous hashing over:

```
programId
assignmentEpoch
finalized Ethereum seed
nodeId
```

The first eligible node is the scheduler and the next N eligible nodes form the executor committee, subject to stake and capacity requirements.

The assignment seed is used to distribute work, not as a source of cryptographic application randomness.

A failed scheduler may be replaced by the next ranked eligible node after a timeout.

8. State and Storage

8.1 State Commitment

A process state commitment is:

```

stateRoot = keccak256(
  domain ||
  vmVersion ||
  codeHash ||
  pc ||
  registersHash ||
  memoryRoot ||
  storeRoot ||
  inboxSeq ||
  outboxSeq ||
  status ||
  units
)

```

The exact serialization is canonical and versioned. Two correct implementations must produce identical bytes before hashing.

8.2 Linear Memory

Linear VM memory is divided into fixed pages. A Merkle tree commits to page hashes. Unallocated pages are represented by protocol-defined zero hashes.

Page-based commitments allow an executor or prover to load only the pages needed by a transition while maintaining a commitment to the entire address space.

8.3 Persistent Store

Applications receive a deterministic key/value store through the runtime ABI. The store is committed separately from transient linear memory and may be implemented using a sparse Merkle tree.

The distinction permits conventional application code to use process memory while durable application records are represented in a state structure suited to proofs and partial retrieval.

8.4 Content-Addressed Blobs

Program binaries, immutable assets, snapshots, and large payloads may be stored using IPFS-compatible content addressing [7].

IPFS is not treated as a persistence guarantee by itself. Assigned storage replicas must pin the data for which they are being paid. The protocol records the content identifier and replication policy; availability is an obligation of the node set.

8.5 Recovery

A node can reconstruct a program from:

1. the latest accepted checkpoint;
2. the code package;
3. a state snapshot matching the checkpoint root;
4. ordered input batches after that checkpoint.

If a snapshot is unavailable but an earlier snapshot and all subsequent inputs remain available, the state may be recomputed.

9. Execution Receipts

After executing a slice or batch, an executor produces a receipt:

```

Receipt {
  programId
  vmVersion
  codeHash
  sequence
  inputRoot
  preStateRoot
  postStateRoot
  outputRoot
  unitsConsumed
  requestedAssurance
  executor
  signature
}

```

The receipt is a statement about one deterministic transition.

Multiple executor signatures over the same receipt form a committee certificate.

A receipt with mismatched roots, units, program version, or sequence is a different statement and cannot be merged into the same certificate.

10. Adaptive Assurance

Chromo defines three assurance classes.

10.1 FAST

FAST is intended for low-value, reversible, or latency-sensitive state.

A FAST transition is accepted by the application once a threshold of its assigned staked executor committee signs the same receipt.

FAST does not claim Ethereum-level correctness. Its security assumption is that the signing threshold of the assigned committee does not collude.

FAST state may continue immediately and may be displayed to users before any Ethereum transaction is submitted.

A signed FAST receipt remains attributable. If a later proof demonstrates that a signer attested to a transition inconsistent with the canonical VM rules, that receipt may be used as evidence in a protocol slashing procedure.

10.2 OPTIMISTIC

An OPTIMISTIC checkpoint posts a transition commitment and proposer bond to Ethereum.

The checkpoint enters a challenge window. If no valid challenge is made before the window ends, the checkpoint becomes settled.

If a challenger disputes the transition, the proposer must produce a validity proof for exactly the disputed transition before a response deadline.

```

proposal
|
+---- no challenge ----> settled
|
+---- challenge
|
|         +---- valid proof ----> settled; challenger bond lost
|         |
|         +---- no valid proof -> rejected; proposer bond slashed

```

This design avoids a custom on-chain RISC-V interactive dispute interpreter. The expensive proof is generated only when a dispute occurs.

The challenger posts a bond to discourage costless griefing. A successful challenger receives a protocol-defined portion of the slashed proposer bond. When the proposer proves that a challenge was false, the challenger bond compensates the proposer/prover for the forced proof cost before any remaining portion is burned or returned to a protocol pool.

10.3 PROVEN

A PROVEN checkpoint includes a validity proof at submission time.

Ethereum accepts the new checkpoint only if the proof verifies against:

```
program code commitment
pre-state root
input commitment
post-state root
VM version
metering result
```

PROVEN is intended for high-value, irreversible, or externally settled actions.

10.4 Assurance Monotonicity

A transition may move from weaker to stronger assurance but never in the opposite direction.

```
FAST -> OPTIMISTIC -> PROVEN
```

A program may continue from FAST state, but an Ethereum action that requires PROVEN assurance can only reference a state whose complete unproven ancestry has been covered by a validity proof.

A PROVEN proof therefore begins at the most recent already-PROVEN state, or at the protocol-approved genesis state, and may cover many execution slices at once. Proving only the final slice on top of an unproven ancestor does not upgrade that ancestor.

If an earlier optimistic ancestor is rejected, descendants that depended upon it are invalidated unless they are re-executed from the last valid ancestor.

11. Validity Proof Backend

The proof system is deliberately modular. Ethereum only requires a verifier contract that can validate the canonical Chromo transition statement.

The reference design uses an existing RISC-V zero-knowledge virtual machine rather than inventing a new proving system [8]. A proof job receives:

```
code or code commitment
canonical pre-state witness
canonical ordered inputs
claimed post-state root
claimed outputs
claimed metering
```

It re-executes the deterministic transition and produces a proof that the public commitment is correct.

An RV32IM proving environment is a natural fit because Chromo's canonical execution machine uses the same base model. The proving adapter must implement the same Chromo ECALL semantics against committed inputs rather than arbitrary host I/O.

Correctness depends on equivalence between normal execution and proof execution. Conformance test vectors are therefore protocol artifacts: every accepted node and prover implementation must produce the same result for the same transition vector.

12. Ethereum Protocol Contracts

The Ethereum layer is intentionally small.

12.1 NodeRegistry

Stores:

```
node address
peer identity
staked amount
capabilities
status
metadata commitment
```

It is the canonical membership set used for assignment.

12.2 StakeManager

Locks the native protocol token used as collateral.

Stake may be slashed for objectively provable violations, including signing an invalid transition that is later disproved, failing a proof-on-challenge obligation after proposing a checkpoint, or equivocation under protocol rules.

Ordinary temporary downtime should primarily reduce assignment eligibility and rewards rather than automatically destroy stake.

12.3 ProgramRegistry

Stores:

```
programId
owner
codeHash
manifest commitment
vmVersion
security policy
latest settled checkpoint
resource account
Ethereum action policy
```

12.4 CheckpointManager

Accepts OPTIMISTIC and PROVEN checkpoint submissions, manages challenge windows, records checkpoint status, and updates the settled state root.

12.5 ProofVerifier

Adapts one or more external proving systems to a common verification interface.

A verifier is identified by a version. Existing programs are not silently migrated to a verifier with different semantics.

12.6 ProgramVault and ActionBridge

Ethereum assets associated with an application are held by a smart-contract vault rather than by executor private keys.

A Chromo program may emit an action intent:

```
ActionIntent {
  programId
  checkpoint
  nonce
  target
  value
  calldataHash
  calldata
  minimumAssurance
}
```

ActionBridge executes the intent only if:

1. the referenced checkpoint has reached the required assurance;
2. the action nonce has not been used;
3. the call is permitted by the program's vault policy;
4. the payload matches the committed output;
5. any value and rate limits are satisfied.

The executor therefore never receives unilateral custody of application assets.

13. Ethereum Reads

A Chromo program cannot synchronously read an executor's local Ethereum RPC and treat the result as deterministic state.

Ethereum information enters as a chain input containing a canonical block reference and requested data.

Policies may require the source block to be:

```
observed
confirmed by K blocks
justified/finalized under Ethereum consensus
proven by a state/storage proof when applicable
```

The program manifest declares the minimum policy for each class of chain input.

This distinction allows a low-latency application to react to recent observations while requiring stronger Ethereum finality for high-value decisions.

14. Deployment and Program Lifecycle

A compatible client performs deployment as a composition of existing tools rather than as a special consensus client.

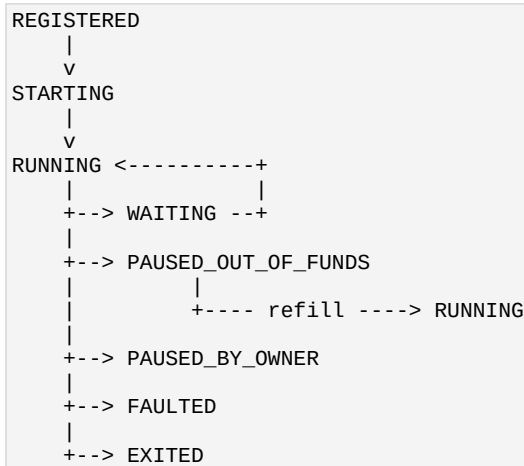
The deployment flow is:

```
source code
  |
  v
C/C++/Rust/RISC-V assembly
  |
  v
standard compiler and linker
  |
  v
static RV32IM ELF + manifest
  |
  v
content-addressed package
  |
  v
deployment quote
  |
  v
fund resource account
  |
  v
Ethereum ProgramRegistry transaction
  |
  v
node assignment
  |
  v
persistent execution
```

Before submitting a deployment, the client obtains a protocol quote containing the minimum initial resource reserve. It reads the user's resource-account balance on Ethereum. If the balance is below the required deployment reserve, the deployment transaction is not submitted or the contract rejects it.

Once registered, assigned nodes retrieve the package by its content identifier, verify the package hashes, construct the initial machine state, and compare the resulting initial state root with the registered commitment.

A program has the following protocol-visible lifecycle:



A process may remain in **WAITING** without consuming compute until a matching input becomes available. The owner may configure a resource sponsor. In that case ordinary users interact by signing application messages and do not need to hold CHR or submit Ethereum transactions. The application resource account pays the execution network.

14.1 Runtime Settlement of Node Fees

Nodes do not receive a token transfer for every instruction.

For each accepted checkpoint, the receipt commits to resource consumption. The protocol computes:

```

charge =
  computeUnits * computePrice
+ storageUnits * storagePrice
+ bandwidthUnits * bandwidthPrice
+ proofFee
  
```

The CheckpointManager settles the charge from the program resource account to the node-service pool according to the accepted receipt.

This produces one economic settlement for many off-chain application actions and prevents a node from charging an arbitrary amount: resource counts are deterministic outputs of the same execution transition that can be replayed or proven.

14.2 Direct Client Interaction

A client may submit messages to any reachable network node. It does not need to maintain a direct connection to the assigned scheduler.

A receiving node verifies the message envelope and propagates it toward the current program committee. The client may request receipts at different stages:

RECEIVED	message accepted by a relay
ORDERED	included in a signed batch
EXECUTED	committee produced matching state receipt
SETTLED	optimistic challenge period completed
PROVEN	validity proof accepted

Applications may map these protocol states into user-facing confirmations appropriate to their domain.

15. Resource Accounting

15.1 Compute Units

Ethereum gas is not reused directly because Chromo execution has different costs.

Every canonical instruction and runtime operation has a deterministic compute-unit weight.

Example conceptual weights:

simple integer instruction	1 CU
memory load/store	2 CU
multiplication/division	3-8 CU
persistent store access	higher fixed + byte cost
cryptographic accelerator	algorithm-specific
message emission	fixed + byte cost
blob operation	byte and availability cost
proof generation	separate proof fee

The exact schedule is a versioned protocol table.

A receipt commits to the number of consumed units, making billing replayable and provable.

15.2 Program Balance

A program maintains a funded resource account on Ethereum.

Execution does not require an Ethereum transaction for every instruction or message. Nodes accrue deterministic claims against the program balance and settle them through accepted checkpoints.

If available prepaid credit falls below the required reserve, the scheduler places the program in PAUSED_OUT_OF_FUNDS state. Anyone authorized by the program may refill it.

15.3 Deployment Cost

Deployment pricing covers:

registry transaction cost
code storage/replication
initial state storage
initial node assignment reserve

It is separate from runtime compute.

Binary size is therefore not used as a proxy for future computation.

16. Native Token and Node Economics

The protocol token is denoted CHR in this paper. The symbol is kept distinct from the term CVM, which refers to the virtual machine.

16.1 Functions

CHR has three protocol functions:

1. node collateral;
2. payment and accounting for execution-network resources;
3. penalties and rewards tied to objectively verifiable network behavior.

CHR is not required to be the unit of account exposed to end users. Wallets, gateways, or application operators may sponsor fees or convert other assets into CHR before funding a program resource account.

16.2 Supply

The conceptual genesis supply is:

Founder treasury	30,000,000 CHR
Foundation treasury	10,000,000 CHR
Network distribution	20,000,000 CHR

Total	60,000,000 CHR

This allocation is economic policy, not part of VM correctness. The protocol should not equate token ownership with unrestricted execution authority or simple one-token-one-vote control over application state.

16.3 Node Stake

A node must lock collateral before becoming eligible.

A fixed floor may be used for spam resistance, but security exposure should scale with assigned responsibility:

```
requiredStake =
  max(baseStake,
    securityFactor * assignedEconomicExposure)
```

A conceptual initial base stake may be 2,000 CHR. It is a parameter rather than a permanent invariant.

16.4 Fees

For a settled execution charge F :

```
node service pool = 0.80 * F
protocol burn      = 0.20 * F
```

The node service pool is distributed according to measured roles performed for the transition: scheduling, execution, proof generation when requested, and storage availability where separately metered.

Challenge rewards are funded primarily from slashed invalid proposer bonds, avoiding a permanent tax on honest execution solely to subsidize challenges.

The 80/20 split is a policy parameter and can be changed without changing execution semantics.

16.5 Staking Yield

The protocol does not guarantee a fixed 4-6 percent APY.

Node return is:

```
nodeReturn =
  execution fees
+ proof fees
+ storage fees
+ temporary network-distribution emissions
- slashing losses
- operating costs
```

Bootstrap emissions from the network distribution reserve may be calibrated toward a target range during early operation, but a quoted APY is not a consensus promise. Long-run security is intended to be paid primarily by real network usage.

17. Security Model

17.1 FAST Security

FAST correctness requires an honest threshold of the assigned executor committee.

An attacker controlling the threshold stake for a particular assignment may sign an invalid FAST state. For this reason, FAST must not be used by an Ethereum vault or another program when the receiving policy requires stronger assurance.

17.2 OPTIMISTIC Security

OPTIMISTIC correctness requires:

1. Ethereum remains secure;
2. invalid checkpoints can be observed;
3. at least one economically motivated challenger can submit a challenge during the window;
4. the validity proof backend is sound;
5. the proposer cannot prevent the proof-verification transaction from reaching Ethereum indefinitely.

The mechanism does not require every executor to replay every program forever.

17.3 PROVEN Security

PROVEN correctness depends on:

1. Ethereum contract execution;
2. soundness of the configured proof system;
3. correctness of the Chromo transition circuit/guest;
4. collision resistance of commitment hashes;
5. availability of the committed input and state witness needed to generate the proof.

17.4 Data Availability

A correct state root is not useful if all data needed to continue execution disappears.

Programs therefore configure a replication factor. Executor and storage nodes are paid to retain:

```
code package
recent snapshots
input batches
outputs required by dependents
```

Availability attestations may affect rewards, but attestations alone do not prove long-term possession. High-value applications should use independent storage replicas and external content-addressed persistence in addition to the active executor committee.

17.5 Censorship and Liveness

A scheduler can delay messages. It cannot forge user signatures.

Users may broadcast to multiple nodes. A scheduler that fails to produce a batch can be replaced. A program may additionally expose an Ethereum forced-inbox path for messages whose inclusion must be censorship-resistant at the cost of Ethereum latency and fees.

The forced inbox is not required for normal operation.

17.6 Determinism Bugs

A mismatch between two conforming VM implementations is a consensus-critical fault.

The protocol therefore versions:

```
VM semantics
syscall ABI
serialization
hash domains
metering schedule
proof adapter
```

Upgrades create a new VM version. A running program remains pinned to its declared version until it explicitly migrates.

18. Example: Persistent Decentralized Application

Consider an online game whose world process runs continuously.

Users submit signed movement messages. The assigned scheduler orders them and executor nodes update the world state without waiting for Ethereum.



The game loop may conceptually be written as:

```

for (;;) {
    msg = chromo_rcv();
    update_world(msg);

    if (high_value_event(msg))
        chromo_checkpoint(PROVEN);
    else if (periodic_boundary())
        chromo_checkpoint(OPTIMISTIC);
    else
        chromo_checkpoint(FAST);
}

```

A user may move, chat, craft items, and participate in the world at execution-network latency. No Ethereum block is required for each action.

If an item is withdrawn as an Ethereum asset, the application emits an `ActionIntent` referencing a checkpoint whose assurance policy is `PROVEN`. The `ActionBridge` verifies the checkpoint and executes the authorized vault action.

Thus the runtime behaves like an application backend while Ethereum remains the final authority for assets and high-value settlement.

19. Why There Is No Chromo Blockchain

A separate execution blockchain would reintroduce a global block sequence, a global consensus mechanism, chain synchronization, fork choice, and a second independent economic-security domain.

Chromo instead treats each application as an independently progressing deterministic process.

Ethereum already provides the shared ledger needed for:

```

node registration
stake and slashing
program ownership
program balances
settled state commitments
proof verification
on-chain asset custody

```

The execution network only needs to agree enough to deliver the assurance selected by a particular transition.

This separation keeps the trusted base small and allows two unrelated applications to use entirely different executor committees and execution rates without competing for the same global block.

20. Implementation Components

The logical protocol intentionally maps to existing components.

Execution ISA

```
RISC-V RV32IM.
```

Compiler toolchain

LLVM/Clang, GNU toolchains, RISC-V ELF psABI.

Networking

libp2p transports, peer identity, discovery, request/response, and pub/sub.

Content addressing

IPFS-compatible CIDs for binaries, snapshots, and large immutable blobs.

Ethereum

Solidity contracts for registry, stake, balances, checkpoints, verifier routing, and vault actions.

Local node state

Any durable embedded database capable of exact key/value persistence; its format is not consensus-critical.

Validity proof

Existing RISC-V zkVM/proving backend through a versioned adapter.

Cryptographic commitments

Keccak-256 for Ethereum-facing protocol commitments unless a proof backend uses an internal hash and exposes a verified mapping to the canonical commitment.

No component above is allowed to define application behavior implicitly. Consensus behavior is defined by the ChromoVM transition specification and canonical serialization.

21. Limitations

Chromo does not make off-chain computation free. Nodes still consume CPU, memory, bandwidth, storage, and proof resources.

FAST execution is not equivalent to Ethereum finality.

An optimistic checkpoint has latency equal to its challenge policy before it becomes settled.

A validity proof can be computationally expensive even when verification on Ethereum is compact.

External information remains an oracle problem. Chromo makes external inputs explicit and deterministic but does not make an untrustworthy data source truthful.

Content addressing identifies data but does not guarantee persistence.

A program that requires every state change to have immediate Ethereum-equivalent assurance may receive little benefit from the fast execution path.

22. Conclusion

ChromoVM separates decentralized application execution from blockchain settlement.

Programs execute as persistent deterministic RISC-V processes on a staked off-chain network. They exchange messages, maintain memory and durable state, and progress independently of Ethereum block production. Ethereum is not replaced; it is used where its properties are strongest: collateral, immutable commitments, dispute settlement, validity verification, and asset custody.

The execution layer is not a blockchain and does not introduce a global replicated ledger. Applications scale independently. Standard RISC-V and existing compiler toolchains avoid the cost of inventing a new CPU architecture, while an ECALL-based runtime provides decentralized storage, messaging, chain interaction, and checkpoint control.

Most importantly, Chromo does not impose one security-cost-latency tradeoff on every application action. FAST, OPTIMISTIC, and PROVEN transitions allow each application to decide where stronger assurance is worth paying for. The intended result is a decentralized runtime in which applications can behave like ordinary continuously running software while retaining a direct path to Ethereum-backed trust when their state or assets require it.

References

- [1] RISC-V International, "RISC-V Unprivileged ISA Specification," ratified specification library, 2026.
<https://docs.riscv.org/reference/isa/v20260120/unpriv/>
- [2] RISC-V International, "RV32I Base Integer Instruction Set," Version 2.1.
<https://docs.riscv.org/reference/isa/v20260120/unpriv/rv32.html>
- [3] RISC-V ELF psABI, "RISC-V ABIs Specification."
<https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>
- [4] LLVM Project, "User Guide for RISC-V Target."
<https://llvm.org/docs/RISCVUsage.html>
- [5] Ethereum.org, "Proof-of-Stake."
<https://ethereum.org/developers/docs/consensus-mechanisms/pos/>
- [6] libp2p Project, protocol documentation.
<https://libp2p.io/>
- [7] IPFS Documentation, "How IPFS Works."
<https://docs.ipfs.tech/concepts/how-ipfs-works/>
- [8] RISC Zero, "zkVM Technical Specification."
<https://dev.risczero.com/api/zkvm/zkvm-specification>